

Knowledge = Agent 的长期记忆系统

根本限制: Agent 每次启动都是白纸——不知道项目做什么、上次做到哪、踩过什么坑、底线在哪。

DDD

结构化领域知识

✓ NOW

Feed

经验流入入口

✓ NOW

Governance

三层治理（下午）

LATER

Distillation

蒸馏进化（下午）

LATER

把 CLAUDE.md 从垃圾场变成图书馆

CLAUDE.md (200 行, 什么都有)

项目描述.....
一堆规则.....
代码规范.....
API 配置.....
IMPORTANT IMPORTANT !!!
(继续加粗、加感叹号.....)

什么都有 = 什么都不突出

Attention 衰减

前 1000 token
attention 最强 //

中段
平稳但弱 //

尾段
可能被忽略 -

DDD: 按需注入正确知识

PRODUCT.md

TECH.md

IMPROVEMENT.md

PROJECT.md

4 份职责分明 · SessionStart 按策略选塞

4 份文档，4 种知识，各司其职

文档	存什么	什么时候注入	类比
PRODUCT.md	业务领域：用户是谁、解决什么问题、核心概念、业务规则	每次 SessionStart	新员工入职手册
TECH.md	技术架构：技术栈、架构决策、不可逆约束、接口契约	每次 SessionStart	架构设计文档
IMPROVEMENT.md	改进方向：已知 tech debt、优先级、禁止事项	涉及重构时	团队 retro 产出
PROJECT.md	项目状态：当前 sprint、进行中任务、阻塞项	每次 SessionStart	站会白板

单一职责： PRODUCT 不写技术细节 · TECH 不写业务逻辑 · IMPROVEMENT 不写正在做的事 · PROJECT 不写历史决策

为错题本写 DDD —— 你来判断放什么

错题本 - 产品知识

用户

K12 学生家长 (代孩子管理错题)

核心概念

- 错题 / 识别 / 知识点 / 复习计划

业务规则

- 一道错题必须关联至少一个知识点
- 识别 **confidence < 0.7** 必须标记"待确认"
- 同一知识点连续 3 次正确 → "已掌握"

哪些是「判断」？

0.7 阈值 —— 为什么不是 0.8? 0.5?

"连续 3 次" —— 为什么不是 5 次?

这些就是品味 = 你选的线 + 理由 + 判定标准

DDD 不是事实罗列 (AI 30 秒能写), 而是判断的记录。

TECH.md 的核心：记录不可逆决策

错题本 — 技术知识

技术栈

React19 · FastAPI · DynamoDB · Bedrock · Lambda

不可逆决策 (ADR)

[ADR-001] DynamoDB 而非 RDS

理由：单表设计适合读多写少

代价：复杂查询能力受限

不可逆：数据模型不同，迁移 = 重写

[ADR-002] Bedrock 而非自建模型

不可逆：prompt 与后处理和 Claude 耦合

ADR 是 TECH.md 最重要的部分

不可逆 = 一旦选了，走回头路成本极高。

Agent 必须知道这些 —— 否则它可能「优化」你的架构决策：

"建议迁移到 PostgreSQL 简化查询"

只记「改了就是重写」的决策 —— 帮你做减法。

改进方向 + 项目状态（最短命的两份文档）

错题本 – 改进方向

优先级

1. 识别准确率 (baseline 未建立)
2. 首次体验 (拍照→识别 < 5s)

已知 Tech Debt

- 错误处理不统一

禁止事项

- 不要动 auth 模块 (重构中)
- 不要添加 ORM (与单表冲突)

错题本 – 项目状态

当前 Sprint

Sprint 3: 核心识别功能

进行中

- [] 拍照上传 API
- [] Bedrock 识别调用
- [x] DynamoDB schema 设计

阻塞

- Bedrock 配额申请中 (周三到)

PRODUCT + TECH = 稳定 (月更, 手动维护)

IMPROVEMENT + PROJECT = 高频 (周/日变, 需自动更新)

Knowledge 不是手动填的表格 —— 它需要自动化输入源

Feed 来源	信号	流入哪里
Session 结束	corrections、决策、发现	corrections.log → 待蒸馏
代码变更	commit diff、PR 描述	PROJECT.md（自动更新状态）
外部系统	CI 结果、deploy 状态、告警	PROJECT.md / IMPROVEMENT.md

课程聚焦：Session 结束时的自动 capture（最核心的 feed）

- 其他 feed 更工程化，超出三天课程范围
- 但概念要清楚：Knowledge 是有多路输入的系统，不是单一文档 —— 多路输入 → Feed → Knowledge 四文档

SessionEnd Hook = Knowledge 的「耳朵」

```
# hooks/on-session-end.sh
#!/bin/bash
# 触发: Claude Code session 结束时

claude --print "Review this session transcript.
Extract:
1. corrections (CORRECTION: ...)
2. decisions w/ rationale (DECISION: ...)
3. new facts (DISCOVERY: ...)
Output ONLY items, one per line.
If none found, output NONE." \
  >> knowledge/corrections.log
```

Corrections

你纠正 agent 的每一处

Decisions

重要设计/架构决策及理由

Discoveries

新发现的事实 (如 rate limit)

关键约束

- 只提取, 不判断 (蒸馏是后续)
- 追加模式 (不覆盖已有 log)
- 格式统一 (蒸馏脚本能解析)

corrections.log = 蒸馏的原料库

为什么不直接把 correction 变成 rule?

- 一次 correction 可能是偶然 (上下文特殊)
- 三次同方向 = pattern → 才值得升级为 rule
- 五次+ 同类、形式各异 → 升级为 principle
- 过早把 correction 变 rule = 过拟合单次事件

它不是 rules —— 不直接被 agent 读取; 它是蒸馏的输入。

corrections.log 生命周期

积累 (capture hook 追加)

↓ 达到阈值 (5-10 条)

蒸馏 (识别 pattern)

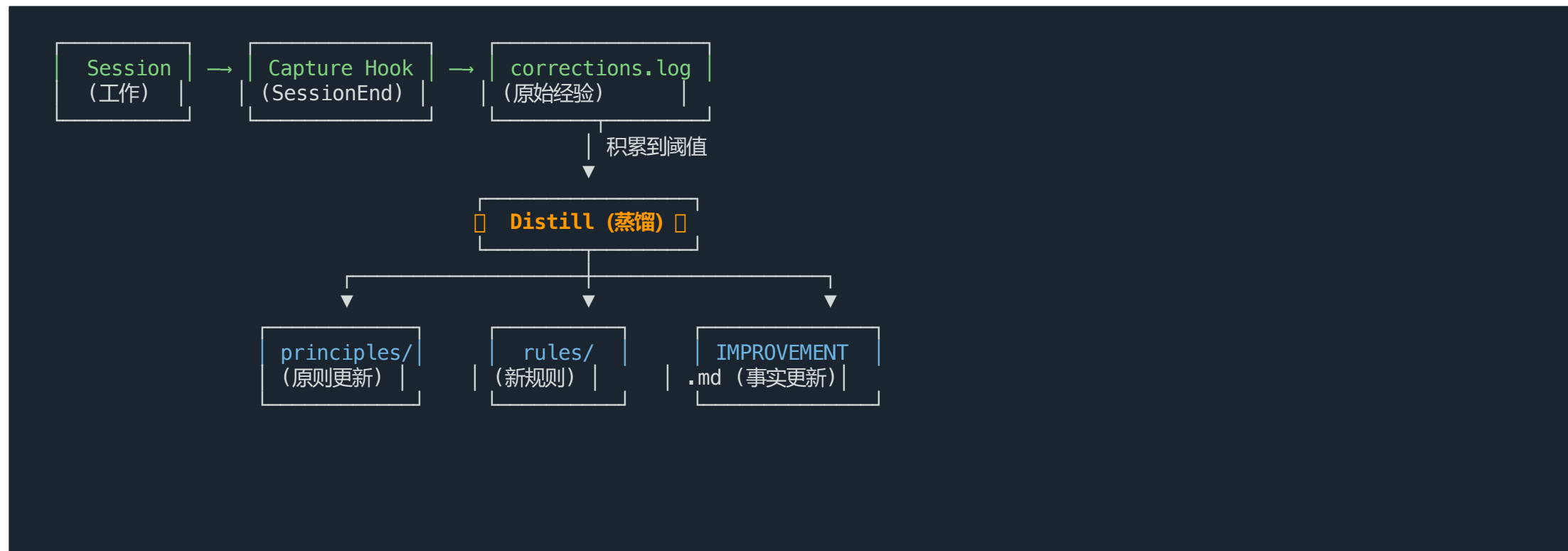
→ 升级为 rule / 发现新 principle

↓ 蒸馏完成

归档 (已处理移入 archive/)

corrections.log 是原料库, 蒸馏是炼油过程 —— 不要把原油直接灌进发动机。

从 session 到 Knowledge —— 信号流全景



上半段 = 今天做的 (capture)

吸入经验，确保不丢失

下半段 = 下午做的 (distill → governance)

处理 → 更新知识 = Knowledge 的一次呼吸

动手时间：为错题本搭 Knowledge 骨架

```
my-agentos/  
├── knowledge/  
│   ├── PRODUCT.md  
│   ├── TECH.md  
│   ├── IMPROVEMENT.md  
│   └── PROJECT.md  
├── hooks/  
│   └── on-session-end.sh  
└── corrections.log (空文件)
```

任务 Checklist

- ✓ 必做：PRODUCT.md + TECH.md 骨架 (Day 2 跑 Engine 马上要用)
- ✓ TECH.md 至少写 2 条 ADR
- ✓ 课后：IMPROVEMENT + PROJECT 留骨架 (Day 2 自然填充)
- ✓ 把 capture hook 配到 Claude Code settings
- ✓ 今天写骨架，不追求完美——血肉明天跑 Engine 时自然长出

判断点提示：

PRODUCT 的 confidence 阈值你选多少？为什么？ · TECH 哪条 ADR 标「不可逆」？标准是什么？

没有标准答案——你的选择 + 你的理由 = 你的品味

同一个错题本，不同的品味

A 同学 · 0.6

"用户可以自己修正，宁可多识别"

后果链

Gate 需覆盖「错误识别导致的数据污染」

B 同学 · 0.8

"错误识别体验太差，宁可少识别"

后果链

需处理「大量待确认堆积」的 UX 问题

C 同学 · 0.7 分级

"0.5-0.7 标黄，< 0.5 不展示"

后果链

实现复杂度上升，需额外 UI 状态

没有谁对谁错 —— 但每个选择都有后果。

品味 = 选择 + 理由 + 对后果的预判 + 愿意承担代价。这些判断就是你 principles / rules 的来源。

文档不更新 = 有毒文档

文档	更新频率	谁负责更新	过期风险
PRODUCT.md	低（需求变更时）	人	低
TECH.md	低（架构决策时）	人	低
IMPROVEMENT.md	中（每 sprint）	人 + 自动	中
PROJECT.md	高（每天工作前）	自动	高

维护策略

- PRODUCT + TECH：手动，纳入「架构决策 checklist」
- IMPROVEMENT：sprint 人更新优先级 + hook 自动追加 debt
- PROJECT：session 开始时自动从 git/task 系统同步

过期检测（Health 预告）

下午做「文档新鲜度检查」：
PROJECT.md 超 3 天未更新 = 健康警报

本节要点回顾

DDD = 4 份结构化文档

PRODUCT / TECH / IMPROVEMENT / PROJECT。核心不是事实罗列，而是判断记录（选择 + 理由 + 代价）。

Feed = Knowledge 的输入管道

Session capture hook = 最核心的 feed；corrections.log = 蒸馏原料，不直接当 rule 用。

维护策略：按更新频率区分手动 / 自动

低频手动、高频自动。过期文档是毒药。

下午预告 ▶

Knowledge(2)：三层治理

corrections 怎么变 principles / rules / gates

Knowledge(3)：Retrieve + Health

文档怎么注入 + 怎么检测健康

DDD = 判断记录 · Feed = 自动捕获 · corrections ≠ rules — 午饭, 13:15 回来。